PROBLEM STATEMENT

Project Title

Order Processing System

Background

Your company processes customer orders through an HTTP API.

However:

- Clients may retry requests due to network issues.
- The payment provider is unreliable.
- Background tasks (invoice/email) must not affect order processing.
- Duplicate orders must be prevented.
- The system must protect itself from repeated payment failures.

You must design and implement a resilient backend system.

Objective

Build a backend system that:

1. Prevents duplicate order creation using Idempotency.
2. Protects the payment service using a manually implemented Circuit Breaker.
3. Ensures background processing does not block the main API.
4. Exposes an HTTP API at:
5. POST /orders

Functional Requirements

1️⃣ Order Creation API

Endpoint

POST /orders

Required Header

Idempotency-Key: <unique_key>

If missing → return:

400 Bad Request

```
{
  "error": "Idempotency-Key required"
}
```

Request Body

```
{
  "customerId": "C101",
  "items": ["item1", "item2"],
  "totalAmount": 2500
}
```

Expected Behavior

If Idempotency-Key is NEW:

1. Create order.
2. Execute payment.
3. Return:

```
{
  "message": "Order Created",
  "order": {
    "_id": "...",
    "customerId": "...",
    "items": [...],
    "totalAmount": 2500,
    "status": "PAID" | "FAILED"
  }
}
```

If Idempotency-Key already exists:

- Return the exact same stored response.
- Do NOT execute payment again.
- Do NOT create a new order.

2️⃣ Idempotency Requirements

- Same Idempotency-Key must always return identical response.
- Payment must not execute twice for the same key.
- Duplicate API calls must not create duplicate orders.

This is explicitly validated in automated testing.

3️⃣ Mock Payment Service

Payment is simulated with random behavior:

- 50% success
- 30% failure
- 20% timeout

The API may return either:

- HTTP 200 (order processed)
- HTTP 500 (internal failure depending on implementation)

Both are acceptable outcomes when payment fails.

4️⃣ Circuit Breaker Rules

The payment call must be wrapped in a Circuit Breaker.

Rules:

- 3 consecutive payment failures → Circuit transitions to OPEN.
- While OPEN:
    - Payment must not execute.
    - API must return fast failure:
    - {
    - "error": "Circuit Open - Try Later"
    - }

The automated tests verify:

- Circuit eventually opens after failures.
- Fast-failure behavior occurs when open.

Cooldown and HALF_OPEN verification is not required for this evaluation version.

5️⃣ Bulkhead Principle (API Level Validation)

Background tasks (invoice/email):

- Must not block order API.
- Even if workers are stopped, API must respond.

For this evaluation:

- API response must return either:
    - HTTP 200
    - HTTP 500

But it must not hang or crash.

MongoDB Order Model

Fields:

- _id
- customerId
- items
- totalAmount
- status (PAID / FAILED)
- createdAt
- updatedAt

Non-Functional Requirements

- API must respond deterministically.
- Idempotency must prevent duplicate payment execution.
- Circuit breaker must protect payment service.
- Background processing must not block API response.
- System must handle concurrent retries safely.

Final Automated Test Coverage

The evaluation system will verify:

1. Basic order creation (HTTP 200 expected).

2. Idempotency returns identical response.
3. Missing Idempotency-Key returns 400.
4. Circuit breaker eventually opens.
5. Fast failure occurs when circuit is OPEN.
6. API still responds (200 or 500) when background workers are stopped.

Only these behaviors are evaluated.